

4. IMPLEMENTASI SISTEM

Bab ini akan menjelaskan mengenai implementasi sistem dari desain sistem yang telah dibahas pada bab sebelumnya. Pada bab ini akan dijelaskan fungsi pada setiap *source code* yang digunakan dalam implementasi program. Tiap *source code* mempunyai fungsi yang berbeda-beda untuk setiap proses peramalan dengan metode yang berbeda.

4.1 Implementasi Perangkat Lunak yang Digunakan

Aplikasi dibangun dengan menggunakan bahasa pemrograman Python untuk penyusunan kode *backend*. *Frontend* dibuat dengan menggunakan bahasa pemrograman HTML, CSS, PHP dan dibantu dengan *framework Bootstrap*. *Database* yang digunakan untuk penyimpanan data adalah *MySQL*. Kemudian, penyusunan kode program memanfaatkan beberapa *library* python, seperti *statsmodels*, *Keras* dan *sklearn*. *Library statsmodels* digunakan untuk membangun model *ARIMA* dan *Holt-Winters Exponential Smoothing*. *Library Keras* digunakan untuk membangun model *LSTM*. *Library sklearn* digunakan untuk normalisasi data menjadi rentang 0-1 untuk metode *LSTM*. *Library sklearn* juga digunakan untuk menghitung nilai evaluasi metrik yaitu nilai *RMSE* dan *MAPE*. *Hardware* yang dipakai saat ini memiliki spesifikasi *processor* Intel Core-i7 dengan tipe 7500U, RAM sebesar 4 GB, serta kartu grafis Nvidia GeForce 930MX sebesar 4 GB memory.

4.2 Implementasi Sistem

Setelah melakukan instalasi *library* yang dibutuhkan, maka langkah selanjutnya adalah membangun aplikasi *website* dengan HTML dan *framework Bootstrap*. Desain sistem yang dibuat pada Bab 3 akan diimplementasikan pada subbab-subbab berikut dan dirincikan ke dalam beberapa segmen program yang daftarnya dapat dilihat pada Tabel 4.1. Implementasi desain sistem dibuat dalam bentuk file php dan python. File yang digunakan akan ditampilkan pada Tabel 4.2 beserta keterangan dan hubungan *activity diagram*.

Tabel 4.1

Pemetaan Segmen Program

Segmen Program	Proses	Keterangan
4.1	Membuat Koneksi ke <i>Database</i>	Membuat koneksi ke <i>database</i> agar dapat melakukan transaksi dengan <i>database</i>
4.2	Menambahkan Data Penjualan Motor	Menambahkan data ke <i>database</i> dengan menyalin data penjualan CSV ke dalam "import.php"
4.3	Melakukan Peramalan	Admin akan memilih jenis motor, periode peramalan serta metode peramalan kemudian menekan tombol " <i>Predict</i> "
4.4	<i>Training</i> dan <i>Testing</i> Model LSTM	Proses <i>training</i> dan <i>testing</i> model LSTM
4.5	<i>Training</i> dan <i>Testing</i> Model ARIMA	Proses <i>training</i> dan <i>testing</i> model ARIMA
4.6	<i>Training</i> dan <i>Testing</i> Model Holt-Winters Exponential Smoothing	Proses <i>training</i> dan <i>testing</i> model Holt-Winters Exponential Smoothing
4.7	Melihat Hasil Peramalan	Pengambilan data dari <i>database</i> dan dilakukan visualisasi ke dalam bentuk tabel dan <i>line chart</i>

Tabel 4.2

Daftar File

Nama File	Activity Diagram	Keterangan
login.php	Gambar 3.10	halaman <i>login</i> untuk admin
home.php	-	halaman utama yang menampilkan fitur website
import.php	-	file untuk <i>import</i> data csv ke dalam <i>database</i>

Nama File	Activity Diagram	Keterangan
data.php	Gambar 3.11	halaman untuk menampilkan data penjualan beserta fitur-fiturnya
index.php	Gambar 3.12	halaman untuk melakukan peramalan
insertdb.php	-	file untuk memasukkan data dari index.php ke <i>database</i>
run.php	-	file untuk menjalankan file <i>python</i>
ramal2.py	-	file <i>python</i> untuk menjalankan model LSTM
ramal3.py	-	file <i>python</i> untuk menjalankan model ARIMA
ramal4.py	-	file <i>python</i> untuk menjalankan model <i>Holt-Winters Exponential Smoothing</i>
hasil.php	-	halaman yang menampilkan hasil peramalan
histori.php	Gambar 3.13	halaman yang menampilkan histori dari peramalan-peramalan yang sudah dilakukan

4.2.1 Membuat Koneksi ke *Database*

Pada bagian ini berguna untuk membuat koneksi ke *database* dengan nama file “conn.php”. File ini akan digunakan dalam halaman lainnya agar dapat melakukan transaksi dengan *database*. *Source code* untuk membuat koneksi ke *database* dapat dilihat pada Segmen Program 4.1.

Segmen Program 4.1

Source Code untuk Membuat Koneksi ke Database

```
<?php
    session_start();
    $host    = 'localhost';
    $user    = 'root';
    $password = '';
    $database = 'dbperamalan';

    $link = mysqli_connect($host, $user, $password);
    $conn = mysqli_select_db($link, $database) or
die(mysqli_error($link));
?>
```

4.2.2 Menambahkan Data Penjualan Motor

Pada bagian ini akan pengguna akan menambahkan data penjualan motor ke dalam *database* dengan nama file “import.php”. Data penjualan dalam bentuk CSV akan di *copy*, kemudian di masukkan ke dalam variabel \$dataA. Tanggal akan di format menjadi bentuk yyyy-mm. Setelah itu jenis motor akan diisi. Kemudian data yang ada ditambahkan ke dalam *database* penjualan. *Source code* untuk menambahkan data penjualan motor dapat dilihat pada Segmen Program 4.2.

Segmen Program 4.2

Source code untuk Menambahkan Data Penjualan Motor

```
<?php
    include "conn.php";
    $dataA="01/15,4
02/15,1
03/15,3
04/15,1
05/15,3
06/15,4
07/15,2
08/15,10
09/15,8
10/15,5
11/15,6
12/15,10
";

    function tanggal($temp)
    {
        $gl=explode("/",$temp);
        if (count($gl)>=2)
        {
            return "20".trim($gl[1])."-".trim($gl[0]);
        }
    }
```

```

    }
    $jenis="VIXION";
    $data=explode("\n",$dataA);
    $tahun="";
    for ($i=0;$i<count($data);$i++)
    {
        print_r($data[$i]);
        echo "<br/>";

        $row=explode(",",$data[$i]);
        if (count($row)>=2)
        {
            $tanggal=tanggal($row[0]);
            $jumlah=$row[1];

            echo $tanggal." : ".$jumlah."<br/>";

            $q="INSERT INTO penjualan (kendaraan,tanggal,jumlah) VALUES
('$jenis','$tanggal','$jumlah)";
            echo $q."<br/>";
            mysqli_query($link,$q);
        }
    }
?>

```

4.2.3 Melakukan Peramalan

Pada Subbab 4.3 ini akan menampilkan *source code* dari file “index.php”. Halaman ini akan menampilkan *form* dimana *admin* akan memilih jenis motor, periode peramalan serta metode peramalan. Setelah admin menekan *button predict*, maka data dari form akan untuk diproses ke file PHP bernama "insertdb.php" yang akan melakukan *error checking*. Jika data sudah berhasil melewati *error checking*, data *request* peramalan akan masuk ke *database*. Setelah itu, file akan menjalankan “run.php”. Dalam “run.php” akan mengeksekusi *file python* untuk melakukan peramalan. *Source code* untuk melakukan peramalan dapat dilihat pada Segmen Program 4.3.

Segmen Program 4.3

Source Code untuk Melakukan Peramalan

```

<form method="POST" action="insertdb.php" style="text-align:center">
    <div class="form-group row">
        <label class="col-sm-4 col-form-label">Jenis
Motor:</label>
        <div class="col-sm-8">
            <div class="input-group mb-3" style="width: 220px;">
                <select name="kendaraan" class="form-select">
                    <option selected value="">-</option>
                </select>
            </div>
        </div>
    </div>
</form>
<?php

```

```

GROUP BY kendaraan";
        $q="SELECT kendaraan FROM penjualan
        $res=mysqli_query($link,$q);
        while ($row=mysqli_fetch_assoc($res))
        {
            ?>
            <option value="<?php echo
$row["kendaraan"];?>">
            <?php
            echo $row["kendaraan"];
            ?>
            </option>
            <?php
        }
        ?>
    </select>
</div>
</div>
</div>
<div class="form-group row">
    <label class="col-sm-4 col-form-label">Bulan:</label>
    <div class="col-sm-8">
        <div class="input-group mb-3" style="width: 220px;">
            <select name="bulan" class="form-select">
                <option selected value="">-</option>
                <option value="1">1</option>
                <option value="3">3</option>
                <option value="6">6</option>
                <option value="12">12</option>
            </select>
        </div>
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-4 col-form-label">Metode:</label>
    <div class="col-sm-8">
        <div class="input-group mb-3" style="width: 220px;">
            <select name="metode" class="form-select">
                <option selected value="">-</option>
                <option value="LSTM">LSTM</option>
                <option value="ARIMA">ARIMA</option>
                <option value="Exponential Smoothing">Exponential
Smoothing</option>
            </select>
        </div>
    </div>
</div>
<div class="form-group">
    <button class="btn btn-block border-0 py-2"
style="background-color:#00394f; color:white; margin-
top:18px">Predict</button>
</div>
</form>

```

4.2.4 Training dan Testing Model LSTM

Dalam subbab ini akan membahas mengenai *source code* file “ramal2.py” untuk melakukan *training* dan *testing* model LSTM dengan data penjualan yang diambil dari *database*. *Training* dan *testing* pada subbab ini akan menggunakan dataset penjualan 6 tahun sebagai *dataset training* dan dataset penjualan 1 tahun terakhir sebagai *dataset testing*. Model LSTM dibangun dengan menggunakan *library Keras*. Pada langkah akhir, model akan dievaluasi menggunakan perhitungan RMSE dan MAPE. Pembuatan kode untuk *Training* dan *Testing* Model LSTM dapat dilihat pada Segmen Program 4.4.

Segmen Program 4.4

Source Code untuk *Training* dan *Testing* Model LSTM

```
import numpy
import matplotlib.pyplot as plt
import pandas
import math
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_percentage_error

numpy.random.seed(7)

import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="dbperamalan"
)

mycursor = mydb.cursor()

def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return numpy.array(dataX), numpy.array(dataY)

mycursor.execute("SELECT * FROM request_ramal WHERE status=0 AND metode='LSTM'")

myresult = mycursor.fetchall()
```

```

for x in myresult:
    mycursor.execute("SELECT * FROM penjualan WHERE kendaraan='"+x[1]+"")
    myresult2 = mycursor.fetchall()
    dataset = []
    datatest = []
    a = 0
    for x2 in myresult2:
        if (x2[2] == "2021-01"):
            a = 1
        if(a == 1):
            datatest.append([x2[3]])
        else:
            dataset.append([x2[3]])
            tanggalakhir = x2[2]

    tahun = int(tanggalakhir[:4])
    bulan = int(tanggalakhir[5:8])
    scaler = MinMaxScaler(feature_range=(0, 1))
    dataset = scaler.fit_transform(dataset)
    datatest = scaler.fit_transform(datatest)

    train_size = int(len(dataset))
    train = dataset[0:train_size,:]

    look_back = 12
    trainX, trainY = create_dataset(train, look_back)
    trainX = numpy.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))

    model = Sequential()
    model.add(LSTM(10,input_shape=(1, look_back)))
    model.add(Dense(1))
    model.compile(loss='mean_squared_error', optimizer='adam')
    model.fit(trainX, trainY, epochs=100, batch_size=1, verbose=2)

    predicted = []
    test = []
    dakhir=trainX[len(trainX)-1]
    dakhir = numpy.reshape(dakhir, (dakhir.shape[0], 1, dakhir.shape[1]))
    datates = scaler.inverse_transform(datatest)
    for xx in range(0,x[3]) :
        print("dakhir")
        print(dakhir)
        trainPredict = model.predict(dakhir)
        print(trainPredict[0][0])
        test.append(datates[xx][0])
        trainPredict2 = scaler.inverse_transform(trainPredict)
        predicted.append(trainPredict2[0][0])
        ha=str(trainPredict2[0][0])
        te = str(datates[xx][0])
        if (bulan == 12):
            tahun = tahun + 1
            bulan = 1
        else:
            bulan = bulan + 1

```



```

    if bulan < 10:
        bulanakhir = str(tahun) + "-0" + str(bulan)
    else:
        bulanakhir = str(tahun) + "-" + str(bulan)
    print(ha)
    q="INSERT INTO detail_request_ramal
(hasil,bulan,id_request,datareal) VALUES
("+ha+",'" +bulanakhir+"',"+str(x[0])+","+te+")"
    mycursor.execute(q)

    for i in range(look_back-1):
        dakhir[0][0][i] = dakhir[0][0][i+1]

    dakhir[0][0][look_back-1] = datatest[xx][0]
    print("dakhir2")
    print(dakhir)

    RMSE = math.sqrt(mean_squared_error(test, predicted))
    print(RMSE)
    MAPE = mean_absolute_percentage_error(test, predicted)
    print(MAPE)
    q="UPDATE request_ramal SET status='1', RMSE='"+str(round(RMSE,4))+"',
MAPE='"+str(round(MAPE,4))+"' WHERE id='"+str(x[0])
    mycursor.execute(q)

mycursor.close()

mydb.commit()
mydb.close()

```

4.2.5 Training dan Testing Model ARIMA

Pada Subbab 4.5 ini akan membahas mengenai *source code* file “ramal3.py” untuk melakukan *training* dan *testing* model ARIMA dengan data penjualan yang diambil dari *database*. *Training* dan *testing* pada subbab ini akan menggunakan *dataset* penjualan 6 tahun sebagai *dataset training* dan *dataset* penjualan 1 tahun terakhir sebagai *dataset testing*. Model LSTM dibangun dengan menggunakan *library statsmodels*. Pada langkah akhir, model akan dievaluasi menggunakan perhitungan RMSE dan MAPE. Pembuatan kode untuk *Training* dan *Testing* Model ARIMA dapat dilihat pada Segmen Program 4.5.

Segmen Program 4.5

Source Code untuk *Training* dan *Testing* Model ARIMA

```

import warnings
from math import sqrt
from pandas import read_csv
from pandas import datetime
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_percentage_error

```

```

import numpy as np
import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="dbperamalan"
)

mycursor = mydb.cursor()

# return RMSE untuk order (p,d,q)
def evaluate_arima_model(X, Y, arima_order):
    #split training dan testing set
    train, test = X, Y
    history = [x for x in train]
    #prediksi
    predictions = list()
    for t in range(len(test)):
        model = ARIMA(history, order=arima_order)
        model_fit = model.fit()
        yhat = model_fit.forecast()[0]
        predictions.append(yhat)
        history.append(test[t])
    # hitung rmse
    rmse = sqrt(mean_squared_error(test, predictions))
    return rmse

# hitung kombinasi p d q terbaik untuk ARIMA model
def evaluate_models(dataset, p_values, d_values, q_values):
    dataset = dataset.astype('float32')
    best_score, best_cfg = float("inf"), None
    for p in p_values:
        for d in d_values:
            for q in q_values:
                order = (p,d,q)
                try:
                    rmse = evaluate_arima_model(dataset, datatest, order)
                    if rmse < best_score:
                        best_score, best_cfg = rmse, order
                        print('ARIMA%s RMSE=%.3f' % (order,rmse))
                except:
                    continue
    print('Best ARIMA%s RMSE=%.3f' % (best_cfg, best_score))
    return best_cfg

mycursor.execute("SELECT * FROM request_ramal WHERE status=0 AND
metode='ARIMA'")

myresult = mycursor.fetchall()

for x in myresult:
    mycursor.execute("SELECT * FROM penjualan WHERE kendaraan='"+x[1]+"'")

```

```

myresult2 = mycursor.fetchall()
dataset = []
datatest = []
a = 0
for x2 in myresult2:
    if (x2[2] == "2021-01"):
        a = 1
    if(a == 1):
        datatest.append(x2[3])
    else:
        dataset.append(x2[3])
        tanggalakhir = x2[2]

tahun = int(tanggalakhir[:4])
bulan = int(tanggalakhir[5:8])

# evaluate parameters
p_values = range(0,3)
d_values = range(0, 3)
q_values = range(0, 3)

warnings.filterwarnings("ignore")
np_array = np.array(dataset)
cfg = evaluate_models(np_array, p_values, d_values, q_values)
print(cfg)
predictions = list()
tes = list()
for xx in range(0,x[3]) :
    model = ARIMA(dataset, order=cfg)
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    tes.append(datatest[xx])
    dataset.append(datatest[xx])
    print('predicted=%f' % (yhat))
    if (bulan == 12):
        tahun = tahun + 1
        bulan = 1
    else:
        bulan = bulan + 1
    if bulan < 10:
        bulanakhir = str(tahun) + "-0" + str(bulan)
    else:
        bulanakhir = str(tahun) + "-" + str(bulan)
    q="INSERT INTO detail_request_ramal
(hasil,bulan,id_request,datareal)
VALUES
("+str(yhat)+", '"+bulanakhir+"', "+str(x[0])+", "+str(datatest[xx])+")"
    mycursor.execute(q)

RMSE = sqrt(mean_squared_error(tes, predictions))
print(RMSE)
MAPE = mean_absolute_percentage_error(tes, predictions)
print(MAPE)

```

```

q="UPDATE request_ramal SET status='1', RMSE='"+str(round(RMSE,4))+"',
MAPE='"+str(round(MAPE,4))+"' WHERE id='"+str(x[0])
mycursor.execute(q)
print(dataset)

mycursor.close()

mydb.commit()
mydb.close()

```

4.2.6 Training dan Testing Model Holt-Winters Exponential Smoothing

Dalam Subbab ini membahas mengenai *source code* file “ramal4.py” untuk melakukan *training* dan *testing* model *Holt-Winters Exponential Smoothing* dengan data penjualan yang diambil dari *database*. *Training* dan *testing* pada subbab ini akan menggunakan *dataset* penjualan 6 tahun sebagai *dataset training* dan *dataset* penjualan 1 tahun terakhir sebagai *dataset testing*. Model *Holt-Winters* dibangun dengan menggunakan *library statsmodels*. Pada langkah akhir, model akan dievaluasi menggunakan perhitungan RMSE dan MAPE. Pembuatan kode untuk *Training* dan *Testing* Model *Holt-Winters Exponential Smoothing* dapat dilihat pada Segmen Program 4.6.

Segmen Program 4.6

Source Code untuk *Training* dan *Testing* Model *Holt-Winters Exponential Smoothing*

```

import pandas as pd
from matplotlib import pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.holtwinters import SimpleExpSmoothing
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from math import sqrt
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_percentage_error
from numpy import array
import numpy as np
import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="dbperamalan"
)

mycursor = mydb.cursor()

mycursor.execute("SELECT * FROM request_ramal WHERE status=0 AND
metode='Exponential Smoothing'")

myresult = mycursor.fetchall()

```

```

for x in myresult:
    mycursor.execute("SELECT * FROM penjualan WHERE kendaraan='"+x[1]+'")
    myresult2 = mycursor.fetchall()
    dataset = []
    datatest = []
    a = 0
    for x2 in myresult2:
        if (x2[2] == "2021-01"):
            a = 1
        if(a == 1):
            datatest.append(x2[3])
        else:
            dataset.append(x2[3])
            tanggalakhir = x2[2]

    print(len(dataset))
    print(len(datatest))
    train, test = dataset, datatest
    print(test)
    tahun = int(tanggalakhir[:4])
    bulan = int(tanggalakhir[5:8])

    #grid search params
    t_params = ['add', 'mul']
    s_params = ['add', 'mul']
    best_score, best_params = float("inf"), None
    for t in t_params:
        for s in s_params:
            fitted_model =
ExponentialSmoothing(train,trend=t,seasonal=s,seasonal_periods=12).fit(
)
            test_predictions = fitted_model.forecast(12)
            print(test_predictions)
            rmse = sqrt(mean_squared_error(test, test_predictions))
            if rmse < best_score:
                best_score, best_params = rmse, [t,s]
            print('Parameter:%s RMSE=%.3f' % ([t, s],rmse))

    print()
    print('Best Parameters%s RMSE=%.3f' % (best_params, best_score))

    print(best_params[0])
    print(best_params[1])
            fitted_model =
ExponentialSmoothing(dataset,trend=best_params[0],seasonal=best_params[
1],seasonal_periods=12).fit()
            test_predictions = fitted_model.forecast(x[3])
            print(test_predictions)

    for xx in range(0,x[3]) :
        if (bulan == 12):
            tahun = tahun + 1
            bulan = 1

```

```

else:
    bulan = bulan + 1
if bulan < 10:
    bulanakhir = str(tahun) + "-0" + str(bulan)
else:
    bulanakhir = str(tahun) + "-" + str(bulan)
    q="INSERT INTO detail_request_ramal
(hasil,bulan,id_request,datareal) VALUES
("+str(test_predictions[xx])+","+str(bulanakhir)+","+str(x[0])+","+str(dat
atest[xx])+")"
    mycursor.execute(q)

RMSE = sqrt(mean_squared_error(datatest, test_predictions))
print(RMSE)
MAPE = mean_absolute_percentage_error(datatest, test_predictions)
print(MAPE)
q="UPDATE request_ramal SET status='1', RMSE='"+str(round(RMSE,4))+"',
MAPE='"+str(round(MAPE,4))+"' WHERE id='"+str(x[0])
mycursor.execute(q)

mycursor.close()

mydb.commit()
mydb.close()

```

4.2.7 Melihat Hasil Peramalan

Pada Subbab 4.7 ini, aplikasi akan menyajikan data yang divisualisasikan ke dalam bentuk tabel dan grafik. Data akan diambil dari *database*. Data hasil peramalan akan ditampilkan ke dalam tabel. Data training penjualan 6 tahun akan ditampilkan dalam grafik serta data hasil peramalan dan data *testing* 1 tahun terakhir. Jenis grafik yang digunakan pada proses ini adalah *line chart*. Dalam melakukan visualisasi grafik dibantu oleh *JavaScript*. *Source code* dari Melihat Hasil Peramalan dapat dilihat pada Segmen Program 4.7.

Segmen Program 4.7

Source Code untuk Melihat Hasil Peramalan

```

<?php
include "conn.php";

$id=mysqli_real_escape_string($link,$_GET["id"]);

$q="SELECT * FROM request_ramal WHERE id='$id'";
$res=mysqli_query($link,$q);
$data=mysqli_fetch_assoc($res);

$motor = $data["kendaraan"];

//show hasil peramalan
$q="SELECT * FROM detail_request_ramal WHERE id_request='$id'";

```

```

$res=mysqli_query($link,$q);

$lbl=[];
$hsl=[];
$real=[];
while ($row=mysqli_fetch_assoc($res))
{
    $lbl[]=$row["bulan"];
    $hsl[]=$row["hasil"];
    $real[]=$row["datareal"];
}

// $hsl2=[];

//show chart
$q="SELECT * FROM penjualan WHERE kendaraan='$motor'";
$res=mysqli_query($link,$q);

$penjualan = array();
$tanggal = array();
while ($row=mysqli_fetch_assoc($res))
{
    if($row["tanggal"] == "2021-01"){
        break;
    }
    array_push($penjualan, $row["jumlah"]);
    array_push($tanggal, $row["tanggal"]);
}
?>

<table class="table table-bordered" style="width: 50%; margin:auto; text-align:center">
    <thead style="background-color:#00394f; color:white">
        <tr>
            <th>Tanggal</th>
            <th>Data Real</th>
            <th>Peramalan</th>
        </tr>
    </thead>
    <?php
        for ($i=0;$i<count($lbl);$i++)
        {
            ?>
            <tr>
                <td>
                    <?php
                        echo $lbl[$i];
                    ?>
                </td>
                <td>
                    <?php
                        echo $real[$i];
                    ?>
                </td>
            </tr>
        }
    </?php>
</table>

```

```

        <td>
            <?php
                echo round($hsl[$i]);
            ?>
        </td>
    </tr>
<?php
}
?>
</table>

<script>
const ctx = document.getElementById('myChart').getContext('2d');
const myChart = new Chart(ctx, {
    type: 'line',
    data: {
        datasets: [
            {
                label: 'Penjualan 2015-2020',
                data: [<?php
                    foreach($penjualan as $i => $p){
                        echo '{x:"' . $tanggal[$i] . '", y:"' . $p . '"},';
                    }
                ?>],
                fill: false,
                borderColor: "#17a2b8",
                tension: 0.1
            },
            {
                label: 'Data real',
                data: [<?php
                    foreach($real as $i => $r){
                        echo '{x:"' . $lbl[$i] . '", y:"' . $r . '"},';
                    }
                ?>],
                fill: false,
                borderColor: "#61b0b7",
                tension: 0.1
            },
            {
                label: 'Hasil peramalan',
                data: [<?php
                    foreach($hsl as $i => $h){
                        echo '{x:"' . $lbl[$i] . '", y:"' . round($h) . '"},';
                    }
                ?>],
                fill: false,
                borderColor: "#f79256",
                tension: 0.1
            }
        ]
    },
    options: {
        scales: {

```



```
        y: {
            beginAtZero: true
        }
    }
});
</script>
```